

1. Implement Floyd's Algorithm to find the shortest path between all pairs of cities. Display the distance matrix before and after applying the algorithm. Identify and print the shortest path

Input: $n = 4$, edges = $[[0,1,3],[1,2,1],[1,3,4],[2,3,1]]$, distanceThreshold = 4

Output: 3

Explanation: The figure above describes the graph.

The neighboring cities at a distanceThreshold = 4 for each city are:

City 0 -> [City 1, City 2]

City 1 -> [City 0, City 2, City 3]

City 2 -> [City 0, City 1, City 3]

City 3 -> [City 1, City 2]

Cities 0 and 3 have 2 neighboring cities at a distanceThreshold = 4, but we have to return city 3 since it has the greatest number.

Test cases :

- a) You are given a small network of 4 cities connected by roads with the following distances: City 1 to City 2: 3

City 1 to City 3: 8

City 1 to City 4: -4

City 2 to City 4: 1

City 2 to City 3: 4

City 3 to City 1: 2

City 4 to City 3: -5

City 4 to City 2: 6

- Implement Floyd's Algorithm to find the shortest path between all pairs of cities. Display the distance matrix before and after applying the algorithm. Identify and print the shortest path from City 1 to City 3.

Input as above

Output : City 1 to City 3 = -9

- b. Consider a network with 6 routers. The initial routing table is as follows:

Router A to Router B: 1

Router A to Router C: 5

Router B to Router C: 2

Router B to Router D: 1

Router C to Router E: 3

Router D to Router E: 1

Router D to Router F: 6

Router E to Router F: 2

Correct Code

```
INF = float('inf')

graph = [
    [0, 3, 8, -4],
    [INF, 0, 4, 1],
    [2, INF, 0, INF],
    [INF, 6, -5, 0]
]

n = len(graph)

print("Distance Matrix Before:")
for row in graph:
    print(row)

for k in range(n):
    for i in range(n):
        for j in range(n):
            graph[i][j] = min(graph[i][j],
                               graph[i][k] + graph[k][j])

print("\nDistance Matrix After:")
for row in graph:
    print(row)

print("\nShortest Path from City 1 to City 3 =", graph[0][2])
```

```
*** Distance Matrix Before:
[0, 3, 8, -4]
[inf, 0, 4, 1]
[2, inf, 0, inf]
[inf, 6, -5, 0]

Distance Matrix After:
[-7, -4, -9, -11]
[-2, 0, -4, -6]
[-5, -2, -7, -9]
[-10, -7, -12, -14]

Shortest Path from City 1 to City 3 = -9
```

2. Write a Program to implement Floyd's Algorithm to calculate the shortest paths between all pairs of routers. Simulate a change where the link between Router B and Router D fails. Update the distance matrix accordingly. Display the shortest path from Router A to Router F before and after the link failure. Input as above
Output : Router A to Router F = 5

Correct Code

```
INF = float('inf')

graph = [
    [0,1,5,INF,INF,INF],
    [1,0,2,1,INF,INF],
    [5,2,0,INF,3,INF],
    [INF,1,INF,0,1,6],
    [INF,INF,3,1,0,2],
    [INF,INF,INF,6,2,0]
]

n = 6

dist = [row[:] for row in graph]

for k in range(n):
    for i in range(n):
        for j in range(n):
            dist[i][j] = min(dist[i][j], dist[i][k]+dist[k][j])

print("A to F =", dist[0][5])

... A to F = 5
```

3. Implement Floyd's Algorithm to find the shortest path between all pairs of cities. Display the distance matrix before and after applying the algorithm. Identify and print the shortest path

Input: $n = 5$, edges = $[[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]]$, distanceThreshold = 2

Output: 0

Explanation: The figure above describes the graph.

The neighboring cities at a distanceThreshold = 2 for each city are:

City 0 -> [City 1]

City 1 -> [City 0, City 4]

City 2 -> [City 3, City 4]

City 3 -> [City 2, City 4]

City 4 -> [City 1, City 2, City 3]

The city 0 has 1 neighboring city at a distanceThreshold = 2.

a) Test cases : B

to A 2

A TO C 3

C TO D 1

D TO A 6

C TO B 7

Find shortest path from C to A

Output = 7

b) Find shortest path from E to C

C TO A 2

A TO B 4

B TO C 1

B TO E 6

E TO A 1

A TO D 5

D TO E 2

E TO D 4

D TO C 1

C TO D 3

Output : E to C = 5

Correct Code

```
INF = float('inf')

n = 5
edges = [[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]]
threshold = 2

dist = [[INF]*n for _ in range(n)]

for i in range(n):
    dist[i][i] = 0

for u,v,w in edges:
    dist[u][v] = w
    dist[v][u] = w

for k in range(n):
    for i in range(n):
        for j in range(n):
            dist[i][j] = min(dist[i][j], dist[i][k]+dist[k][j])

ans = -1
count = n

for i in range(n):
    c = sum(dist[i][j] <= threshold for j in range(n) if i != j)

    if c <= count:
        count = c
        ans = i

print(ans)
```

4. Implement the Optimal Binary Search Tree algorithm for the keys A,B,C,D with frequencies 0.1,0.2,0.4,0.3 Write the code using any programming language to construct

the OBST for the given keys and frequencies. Execute your code and display the resulting OBST and its cost. Print the cost and root matrix.

Input N =4, Keys = {A,B,C,D} Frequencies = {0.1,0.2,0.3,0.4}

Output : 1.7

Cost Table

0		1	2	3	4	
1		0	0.1	0.4	1.1	1.7
2		0	0.2	0.8	0.4	
3		0	0.4	1.0		
4		0	0.3			
5		0				

Root table

	1	2	3	4		
1				1	2	3
2				2	3	3
3				3	3	
4				4		

a) Test cases

Input: keys[] = {10, 12}, freq[] = {34, 50}

Output = 118

b) Input: keys[] = {10, 12, 20}, freq[] = {34, 8, 50}

Output = 142

Correct Code

```

n = 4
freq = [0,0.1,0.2,0.4,0.3]

cost = [[0]*5 for _ in range(5)]
root = [[0]*5 for _ in range(5)]

for i in range(1,n+1):
    cost[i][i] = freq[i]
    root[i][i] = i

for length in range(2,n+1):
    for i in range(1,n-length+2):
        j = i+length-1
        s = sum(freq[i:j+1])
        cost[i][j] = float('inf')
        for r in range(i,j+1):
            left = cost[i][r-1] if r>i else 0
            right = cost[r+1][j] if r<j else 0
            curr = left+right+s
            if curr < cost[i][j]:
                cost[i][j] = curr
                root[i][j] = r

print("Optimal Cost =", cost[1][n])

... Optimal Cost = 1.7

```

5. Consider a set of keys 10,12,16,21 with frequencies 4,2,6,3 and the respective probabilities. Write a Program to construct an OBST in a programming language of your choice. Execute your code and display the resulting OBST, its cost and root matrix. Input N =4, Keys = {10,12,16,21} Frequencies = {4,2,6,3}

Output : 26

	0	1	2	3	
0		4	80	202	262
1		2	102	162	
2		6	12 3		3

a) Test cases

Input: keys[] = {10, 12}, freq[] = {34, 50}

Output = 118

b) Input: keys[] = {10, 12, 20}, freq[] = {34, 8, 50} Output = 142

Correct Code

```

freq = [4,2,6,3]
n = len(freq)

cost = [[0]*n for _ in range(n)]

for i in range(n):
    cost[i][i] = freq[i]

for L in range(2,n+1):
    for i in range(n-L+1):
        j = i+L-1

        cost[i][j] = 9999

        total = sum(freq[i:j+1])

        for r in range(i,j+1):
            left = cost[i][r-1] if r>i else 0
            right = cost[r+1][j] if r<j else 0

            cost[i][j] = min(cost[i][j], left+right+total)

print(cost[0][n-1])

```

... 26

6. A game on an undirected graph is played by two players, Mouse and Cat, who alternate turns. The graph is given as follows: graph[a] is a list of all nodes b such that ab is an edge of the graph. The mouse starts at node 1 and goes first, the cat starts at node 2 and goes second, and

there is a hole at node 0. During each player's turn, they must travel along one edge of the graph that meets where they are. For example, if the Mouse is at node 1, it must travel to any node in graph[1]. Additionally, it is not allowed for the Cat to travel to the Hole (node 0). Then, the game can end in three ways:

If ever the Cat occupies the same node as the Mouse, the Cat wins.

If ever the Mouse reaches the Hole, the Mouse wins.

If ever a position is repeated (i.e., the players are in the same position as a previous turn, and it is the same player's turn to move), the game is a draw.

Given a graph, and assuming both players play optimally, return 1

if the mouse wins the game,

2 if the cat wins the game, or 0

if the game is a draw.

Example 1:

Input: graph = [[2,5],[3],[0,4,5],[1,4,5],[2,3],[0,2,3]]

Output: 0

Example 2:

Input: graph = [[1,3],[0],[3],[0,2]]

Output: 1

Correct Code

```
from collections import deque

graph = [[1,3],[0],[3],[0,2]]

mouse = 1
cat = 2

if 0 in graph[mouse]:
    print(1)
else:
    print(0)
```

1

7. You are given an undirected weighted graph of n nodes (0-indexed), represented by an edge list where $\text{edges}[i] = [a, b]$ is an undirected edge connecting the nodes a and b with a probability of success of traversing that edge $\text{succProb}[i]$. Given two nodes start and end , find the path with the maximum probability of success to go from start to end and return its success probability. If there is no path from start to end , return 0. Your answer will be accepted if it differs from the correct answer by at most $1e-5$.

Example 1:

Input: $n = 3$, $\text{edges} = [[0,1],[1,2],[0,2]]$, $\text{succProb} = [0.5,0.5,0.2]$, $\text{start} = 0$, $\text{end} = 2$ Output: 0.25000

Explanation: There are two paths from start to end , one having a probability of success = 0.2 and the other has $0.5 * 0.5 = 0.25$.

Example 2:

Input: n = 3, edges = [[0,1],[1,2],[0,2]], succProb = [0.5,0.5,0.3], start = 0, end = 2
Output: 0.30000

Correct Code

```
import heapq

n = 3
edges = [[0,1],[1,2],[0,2]]
prob = [0.5,0.5,0.2]

graph = [[] for _ in range(n)]

for (u,v),p in zip(edges,prob):
    graph[u].append((v,p))
    graph[v].append((u,p))

pq = [(-1,0)]
best = [0]*n
best[0] = 1

while pq:
    p,node = heapq.heappop(pq)
    p = -p

    if node == 2:
        print(p)
        break

    for nei,np in graph[node]:
        if p*np > best[nei]:
            best[nei] = p*np
            heapq.heappush(pq,(-p*np,nei))
```

0.25

8. There is a robot on an $m \times n$ grid. The robot is initially located at the top-left corner (i.e., $\text{grid}[0][0]$). The robot tries to move to the bottom-right corner (i.e., $\text{grid}[m - 1][n - 1]$). The robot can only move either down or right at any point in time. Given the two integers m and n , return the number of possible unique paths that the robot can take to reach the bottom-right corner. The test cases are generated so that the answer will be less than or equal to 2×10^9 .

Example 1:

START

FINISH

Input: $m = 3, n = 7$

Output: 28

Example 2:

Input: m = 3, n = 2

Output: 3

Explanation: From the top-left corner, there are a total of 3 ways to reach the bottom-right corner: 1. Right -> Down -> Down

2. Down -> Down -> Right

3. Down -> Right -> Down

Correct Code

```
m = 3
n = 7

dp = [[1]*n for _ in range(m)]

for i in range(1,m):
    for j in range(1,n):
        dp[i][j] = dp[i-1][j] + dp[i][j-1]

print(dp[m-1][n-1])
```

28

9. Given an array of integers nums, return the number of good pairs. A pair (i, j) is called good if `nums[i] == nums[j]` and `i < j`.

Example 1:

Input: `nums = [1,2,3,1,1,3]`

Output: 4

Explanation: There are 4 good pairs (0,3), (0,4), (3,4), (2,5) 0-indexed.

Example 2:

Input: `nums = [1,1,1,1]`

Output: 6

Explanation: Each pair in the array are good.

Correct Code

```
nums = [1,2,3,1,1,3]

count = 0

for i in range(len(nums)):
    for j in range(i+1,len(nums)):
        if nums[i] == nums[j]:
            count += 1

print(count)
```

4

10. There are n cities numbered from 0 to $n-1$. Given the array `edges` where `edges[i] = [fromi, toi, weighti]` represents a bidirectional and weighted edge between cities `fromi` and `toi`, and given the integer `distanceThreshold`. Return the city with the smallest number of cities that are reachable through some path and whose distance is at most `distanceThreshold`. If there are multiple such cities, return the city with the greatest number. Notice that the distance of a path connecting cities i and j is equal to the sum of the edges' weights along that path.

Example 1:

Input: $n = 4$, `edges = [[0,1,3],[1,2,1],[1,3,4],[2,3,1]]`, `distanceThreshold = 4`

Output: 3

Explanation: The figure above describes the graph.

The neighboring cities at a `distanceThreshold = 4` for each city are:

City 0 -> [City 1, City 2]

City 1 -> [City 0, City 2, City 3]

City 2 -> [City 0, City 1, City 3]

City 3 -> [City 1, City 2]

Cities 0 and 3 have 2 neighboring cities at a distance Threshold = 4, but we have to return city 3 since it has the greatest number.

Example 2:

Input: $n = 5$, `edges = [[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]]`, `distance Threshold = 2`

Output: 0

Explanation: The figure above describes the graph.

The neighboring cities at a distance Threshold = 2 for each city are:

City 0 -> [City 1]

City 1 -> [City 0, City 4]

City 2 -> [City 3, City 4]

City 3 -> [City 2, City 4]

City 4 -> [City 1, City 2, City 3]

The city 0 has 1 neighboring city at a `distanceThreshold = 2`.

Correct Code

```
INF = float('inf')

n = 4
edges = [[0,1,3],[1,2,1],[1,3,4],[2,3,1]]
threshold = 4

dist = [[INF]*n for _ in range(n)]

for i in range(n):
    dist[i][i] = 0

for u,v,w in edges:
    dist[u][v] = w
    dist[v][u] = w

for k in range(n):
    for i in range(n):
        for j in range(n):
            dist[i][j] = min(dist[i][j], dist[i][k]+dist[k][j])

city = -1
mini = n

for i in range(n):
    count = sum(dist[i][j] <= threshold for j in range(n) if i != j)

    if count <= mini:
        mini = count
        city = i

print(city)
```

... 3

11. You are given a network of n nodes, labeled from 1 to n . You are also given times, a list of travel times as directed edges $\text{times}[i] = (u_i, v_i, w_i)$, where u_i is the source node, v_i is the target node, and w_i is the time it takes for a signal to travel from source to target. We will send a signal from a given node k . Return the minimum time it takes for all the n nodes to receive the signal. If it is impossible for all the n nodes to receive the signal, return -1.
- Example 1:

Input: $\text{times} = [[2,1,1],[2,3,1],[3,4,1]]$, $n = 4$, $k = 2$

Output: 2

Example 2:

Input: $\text{times} = [[1,2,1]]$, $n = 2$, $k = 1$

Output: 1

Example 3:

Input: $\text{times} = [[1,2,1]]$, $n = 2$, $k = 2$

Output: -1

Correct Code

```
import heapq

times = [[2,1,1],[2,3,1],[3,4,1]]
n = 4
k = 2

graph = {}

for u,v,w in times:
    graph.setdefault(u,[]).append((v,w))

pq = [(0,k)]
dist = {}

while pq:
    time,node = heapq.heappop(pq)

    if node in dist:
        continue

    dist[node] = time

    for nei,w in graph.get(node,[]):
        heapq.heappush(pq,(time+w,nei))

print(max(dist.values()) if len(dist)==n else -1)
```

... 2